PySpark Syntax Cheat Sheet

Best-practice edition • F.col everywhere • Works in Palantir Foundry

Imports & Skeleton

```python
# One namespace: F / T
from pyspark.sql import functions as F, types as T
from pyspark.sql import Window

# Foundry transform skeleton
from transforms.api import transform_df, Input, Output

@transform_df(
    Output("/proj/clean"),
    src=Input("/proj/raw"),
)
def compute(src):
    return src.filter(F.col("age") > 25)
```

Tip: Never import *: it shadows built-in sum / max / min / round

Inspect

```python
df.show(5, truncate=False)  # 5 rows, untruncated
df.printSchema()            # columns & types
df.columns                  # list of names
df.count()                  # triggers a job
df.describe("salary").show()  # basic stats
df.explain()                # physical plan
```

Tip: count() / collect() / toPandas() are actions; be careful on big data

Select & Add Columns

```python
# select + expression + alias
df.select(
    F.col("name"),
    (F.col("salary") * 1.1).alias("new_salary"),
)

# add or overwrite a column
df = df.withColumn("bonus", F.col("salary") * 0.15)

# constant column via F.lit
df = df.withColumn("src", F.lit("hr"))

# keep all columns and append
df.select(
    F.col("*"),
    (F.col("a") + F.col("b")).alias("c"),
)
```

Tip: Prefer one select over long withColumn chains

Drop, Rename, Cast

```python
# drop columns
df = df.drop("tmp1", "tmp2")

# rename
df = df.withColumnRenamed("old", "new")

# cast
df = df.withColumn("age", F.col("age").cast("int"))
df = df.withColumn("d", F.col("d").cast(T.DateType()))

# rename all (e.g. lowercase)
df = df.toDF(*[c.lower() for c in df.columns])
```

Filter

```python
df.filter(F.col("age") > 25)

# AND is &, parenthesize each
df.filter(
    (F.col("age") > 25) & (F.col("city") == "NYC")
)

# OR is |, NOT is ~
df.filter((F.col("a") == 1) | ~F.col("flag"))

# isin beats chained ORs
df.filter(F.col("dept").isin("Sales", "HR"))

df.filter(F.col("age").between(18, 65))   # inclusive
df.filter(F.col("name").like("A%"))       # wildcard
df.filter(F.col("name").rlike("^A.*n$"))  # regex
df.filter(F.col("email").isNotNull())     # not null

df.dropDuplicates(["email"])  # dedupe by columns
df.distinct()                 # dedupe full rows
```

Tip: Python and / or / not do not work; use & | ~

Conditionals & NULLs

```python
# CASE WHEN: always add otherwise, else NULL
df.withColumn("level",
    F.when(F.col("sal") >= 100000, "A")
     .when(F.col("sal") >= 50000, "B")
     .otherwise("C"))

# first non-null value
F.coalesce(F.col("a"), F.col("b"), F.lit(0))

# fill nulls
df.fillna({"age": 0, "name": "n/a"})

# drop rows with nulls
df.dropna(subset=["email"])

# null-safe equality
F.col("a").eqNullSafe(F.col("b"))
```

Tip: NULL == NULL yields NULL, not True

Strings

```
# trim + lowercase
F.lower(F.trim(F.col("email")))

# 3 chars from pos 1 (1-based)
F.substring(F.col("s"), 1, 3)

# concat: NULL if any input is NULL
F.concat(F.col("a"), F.lit("-"), F.col("b"))

# concat_ws: skips NULLs
F.concat_ws("-", F.col("a"), F.col("b"))

# split, take first part
F.split(F.col("s"), " ").getItem(0)

# keep digits only
F.regexp_replace(F.col("p"), "[^0-9]", "")

# regex extract (group 1)
F.regexp_extract(F.col("e"), "@(.*)$", 1)

F.lpad(F.col("id"), 5, "0")   # left-pad zeros
F.length(F.col("name"))       # length
F.col("s").contains("x")      # contains
```

Dates & Timestamps

```
# string → date / timestamp
F.to_date(F.col("s"), "yyyy-MM-dd")
F.to_timestamp(F.col("s"), "yyyy-MM-dd HH:mm:ss")

# format as string
F.date_format(F.col("d"), "MM/dd/yyyy")

# add / subtract
F.date_add(F.col("d"), 7)     # +7 days
F.date_sub(F.col("d"), 7)     # -7 days
F.add_months(F.col("d"), 3)   # +3 months

# difference (end, start)
F.datediff(F.col("end"), F.col("start"))       # days
F.months_between(F.col("end"), F.col("start")) # months

# extract parts
F.year(F.col("d"))
F.month(F.col("d"))
F.dayofmonth(F.col("d"))
F.dayofweek(F.col("d"))   # 1=Sunday

# truncate to month start
F.trunc(F.col("d"), "month")
F.date_trunc("month", F.col("ts"))

F.current_date()
F.current_timestamp()
```

Math

```
F.round(F.col("x"), 2)     # round half up
F.floor(F.col("x"))        # floor
F.ceil(F.col("x"))         # ceiling
F.abs(F.col("x"))          # absolute value
F.sqrt(F.col("x"))         # square root
F.pow(F.col("x"), 2)       # power
F.log(F.col("x"))          # natural log

# row-wise max / min across columns
F.greatest(F.col("a"), F.col("b"))
F.least(F.col("a"), F.col("b"))
```

Tip: Use F.max / F.min to aggregate rows; greatest / least to compare columns

Aggregation

```
agg = (
    df.groupBy(F.col("dept"))
    .agg(
        F.count(F.lit(1)).alias("n"),          # rows
        F.countDistinct(F.col("id")).alias("n_id"),
        F.sum(F.col("salary")).alias("total"),
        F.avg(F.col("salary")).alias("avg"),
        F.max(F.col("salary")).alias("max"),
        F.collect_set(F.col("name")).alias("names"),
    )
)

# HAVING = filter after agg
agg.filter(F.col("total") > 500000)

# conditional aggregation
F.sum(F.when(F.col("st") == "ok", 1).otherwise(0))

# pivot: listing values is faster
(df.groupBy("dept")
   .pivot("year", [2024, 2025])
   .agg(F.sum(F.col("rev"))))

# whole-table aggregate
df.agg(F.max(F.col("ts")).alias("latest"))
```

Joins & Union

```
# same-name key: no duplicate column
a.join(b, on="id", how="inner")
a.join(b, on=["id", "date"], how="left")

# different names: alias first, then F.col
a = a.alias("a")
b = b.alias("b")
a.join(b, F.col("a.id") == F.col("b.user_id"), "left")

# how options
# inner  left  right  outer  cross
# left_semi  left_anti

# broadcast the small table
a.join(F.broadcast(small), on="id", how="left")

# stack rows, aligned by column name
a.unionByName(b, allowMissingColumns=True)
```

Tip: Always set how; avoid right join, swap the tables instead. Duplicate names after a join cause ambiguity errors

Window Functions

```
w = (
    Window.partitionBy(F.col("dept"))
    .orderBy(F.col("sal").desc())
)

# ranking
F.row_number().over(w)   # unique 1,2,3
F.rank().over(w)         # ties leave gaps
F.dense_rank().over(w)   # ties, no gaps

# Top 3 per group
(df.withColumn("rn", F.row_number().over(w))
   .filter(F.col("rn") <= 3))

# previous / next row
F.lag(F.col("x"), 1).over(w)
F.lead(F.col("x"), 1).over(w)

# running total (row-based frame)
w2 = (
    Window.partitionBy(F.col("uid"))
    .orderBy(F.col("d"))
    .rowsBetween(
        Window.unboundedPreceding,
        Window.currentRow,
    )
)
F.sum(F.col("amt")).over(w2)

# last 7 days: value-based, in seconds
w3 = (
    Window.partitionBy(F.col("uid"))
    .orderBy(F.col("ts").cast("long"))
    .rangeBetween(-6 * 86400, 0)
)
```

Arrays & Structs

```
F.explode(F.col("tags"))          # one row per element
F.explode_outer(F.col("tags"))    # keeps empty / NULL
F.size(F.col("tags"))             # array length
F.array_contains(F.col("tags"), "x")   # contains?
F.collect_list(F.col("x"))        # aggregate into array
F.struct(F.col("a"), F.col("b")) # pack into a struct
F.col("addr.city")                # struct field
F.col("m")["key"]                 # map value
```

Sort & Sample

```
df.orderBy(F.col("a").desc(), F.col("b").asc())

# nulls last
df.orderBy(F.col("a").desc_nulls_last())

df.limit(10)                          # take 10 rows
df.sample(fraction=0.1, seed=42)    # approx. 10%
```

Tip: limit alone gives no order; orderBy first for a real Top N

Output, SQL & Logging

```
# Foundry: just return the DataFrame

# generic Spark write
df.write.mode("overwrite").parquet("/out/path")

# SQL interop
df.createOrReplaceTempView("emp")
spark.sql(
    "SELECT dept, AVG(sal) FROM emp GROUP BY dept"
)

# Python logging
import logging
log = logging.getLogger(__name__)
log.info("rows=%s", df.count())
```

End-to-end Example

```
# top-3 paid active staff per dept
@transform_df(
    Output("/proj/top_paid"),
    emp=Input("/proj/emp"),
)
def compute(emp):
    w = (
        Window.partitionBy(F.col("dept"))
        .orderBy(F.col("sal").desc())
    )
    return (
        emp
        .filter(F.col("active"))
        .withColumn("sal", F.col("sal").cast("double"))
        .withColumn("rn", F.row_number().over(w))
        .filter(F.col("rn") <= 3)
        .select(
            F.col("dept"), F.col("name"), F.col("sal"),
        )
    )
```

Best-Practice Checklist

1. Reference columns with F.col(...); use the F / T namespaces; never import *
2. Prefer a single select over long withColumn chains
3. End every when-chain with otherwise, or unmatched rows become NULL
4. Use & | ~ for boolean logic and parenthesize each condition
5. Always set how in joins; broadcast small tables; alias to avoid duplicate names
6. Filter early and select only needed columns to shrink shuffles
7. Prefer built-in functions; avoid Python UDFs (slow)
8. Avoid collect() / toPandas() on large data; count() triggers a job
9. In production declare the schema explicitly; avoid inferSchema
10. Remember NULL semantics: NULL == NULL is NULL; use eqNullSafe